

MTH00050 – Combinatorial Mathematics

Lecture 4: Recurrences

Lecturer: Bùi Văn Thạch

Lab instructors: Nguyễn Ngọc Toàn, Trần Thị Thảo Nhi, Lê Đức Khoan

{bvthach,nntoan,tttnhi,ldkhoan}@fit.hcmus.edu.vn

(Tentative) Schedule 2026

No.	Date	Location	Topic
1	13/01 16/01	C24 C24/E302	Introduction, Sets, Proofs
2	20/01 23/01	C24 C24/E302	General counting methods
3	27/01 30/01	C24 C24/E302	Inclusion-exclusion principle
4	03/02 06/02	C24 C24/E302	Recurrence relations
5	03/03 06/03	C24 C24/E302	Generating functions
			Mid-term test

No.	Date	Location	Topic
6	10/03 13/03	C24 C24/E302	Graphs (Part I)
7	17/03 20/03	C24 C24/E302	Graphs (Part II)
8	24/03 27/03	C24 C24/E302	Path problems
9	31/03 03/04	C24 C24/E302	Tree problems
10	07/04 10/04	C24 C24/E302	Network flows (Part I)
11	14/04 17/04	C24 C24/E302	Network flows (Part II)
			Final exam

Course topics

0. Introduction
1. Set and counting
2. General counting methods for arrangements and selections
3. Inclusion-exclusion principle
4. Recurrence relations
5. Generating functions
6. Graphs
7. Tree problems
8. Path problems
9. Network flows

Goals

1. To help students able to analyze recursive algorithms by using recurrence techniques.

Outline

1. Substitution method
2. Recursion trees
3. Characteristic equation
4. The Master theorem

Outline

1. Substitution method
2. Recursion trees
3. Characteristic equation
4. The Master theorem

Definition

The substitution method for solving recurrences has two parts.

1. Guess the correct answer.
2. Prove by induction that your guess is correct.

Example: Merge Sort

Analyze the complexity of the merge sort algorithm

```
struct Node {  
    int key;  
    Node* pNext;  
};  
  
struct ListInt{  
    Node first;  
    Node last;  
    int size;  
} LIST;
```

```
void mergeSort(LIST list) {  
    if (list->size == 1)  
        return;  
  
    first_size = floor(list->size/2);  
  
    first_list = LIST(list->first,  
get_node_at_position(list, first_size),  
first_size);  
    second_list =  
LIST(get_node_at_position(list, first_size+1),  
list->last, list->size-first_size);  
  
    mergeSort(first_list);  
    mergeSort(second_list);  
    LIST = mergeTwoSortedLists(first_list,  
second_list);  
}
```


Example 1: Merge Sort: given complexities

1. `get_node_at_position(list, pos)`
 - o runtime $O(pos) = O(\text{list} \rightarrow \text{size})$
2. `mergeTwoSortedLists(first_list, second_list)`
 - o runtime $O(\text{first_list} \rightarrow \text{size} + \text{second_list} \rightarrow \text{size})$
3. `List = LIST(first_node, last_node, size)`
 - o runtime $O(1)$

Example 1: Merge Sort: preliminary analyses

Analyze the complexity of the merge sort algorithm

```
struct Node {
    int key;
    Node* pNext;
};
```

```
struct ListInt{
    Node first;
    Node last;
    int size;
} LIST;
```

1

2

$O(1)$

$O(n)$

$T(n_1)$
 $T(n - n_1)$

$\Theta(n)$

```
void mergeSort(LIST list)
    if (list->size == 1)
        return;

    first_size = floor(list->size/2);

    first_list = LIST(list->first,
get_node_at_position(list, first_size),
first_size);
    second_list =
LIST(get_node_at_position(list, first_size+1),
list->last, list->size-first_size);

    mergeSort(first_list);
    mergeSort(second_list);
    LIST = mergeTwoSortedLists(first_list,
second_list);
}
```

$T(n)$, where $n = \text{list} \rightarrow \text{size}$

Example 1: Merge Sort: Analysis (1/3)

1. Let $T(n)$ be the complexity of the merge sort, where n is the number of items in the list.

2. We have:

$$\begin{aligned} T(n) \\ = 1 + 2 + O(1) + O(n) + T(n_1) + T(n - n_1) + \Theta(n) = T(n_1) + T(n - n_1) + \Theta(n) \end{aligned}$$

3. Without loss of generality, assume $n = 2^k$ for some positive integer k . (Why?)
 $\rightarrow n_1 = n/2$.

Example 1: Merge Sort: Analysis (2/3)

1. Then

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

2. Guess: $T(n) = O(n \log n)$.

3. Proof: Use induction on n to show that there exist c and n_0 for which $T(n) \leq cn \log n$ for all $n \geq n_0$.

Example 1: Merge Sort: Analysis (3/3)

Assume $T(m) \leq cm \log m$ for all $m < n$ to prove that $T(n) \leq cn \log n$. We get

$$T(n) = 2T(n/2) + n$$

$$\leq 2 \cdot c \left(\frac{n}{2}\right) \log \left(\frac{n}{2}\right) + n = cn \log n - cn + n$$

$$\leq cn \log n, \text{ when } c \geq 1.$$

Example 1: Merge Sort: fallacy

1. Assume $T(n) = O(n)$. Then assume $T(m) \leq cm$ for all $m < n$ to prove that $T(n) \leq cn$. We get

$$\begin{aligned} T(n) &= 2T(n/2) + n \\ &\leq 2 \cdot c \left(\frac{n}{2}\right) + n = cn + n \end{aligned}$$

2. It's impossible to prove $T(n) \leq cn$ because of the extra term $+ n$.

Example 2: Even correct guesses can lead to dead end (1/2)

1. Estimate $T(n)$ in terms of a polynomial of n .

$$T(n) = 7T(n/2) + n^2$$

Solution:

1. Guess: $T(n) \leq cn^{\log_2 7} = O(n^{\log_2 7})$.
2. Assume $T(m) \leq cn^{\log_2 7}$ for all $m < n$ to prove that $T(n) \leq cn^{\log_2 7}$. We get

$$T(n) = 7T(n/2) + n^2 \leq 7 \cdot c \left(\frac{n}{2}\right)^{\log_2 7} + n^2 = cn^{\log_2 7} + n^2 > \textcolor{red}{cn^{\log_2 7}}$$

Example 2: Even correct guesses can lead to dead end (2/2)

1. Remedy: set $T(n) \leq cn^{\log_2 7} + an^2$ (to get rid of n^2)

2. We get

$$\begin{aligned} T(n) &= 7T(n/2) + n^2 \leq 7 \left(c \left(\frac{n}{2} \right)^{\log_2 7} + a \left(\frac{n}{2} \right)^2 \right) + n^2 \\ &= cn^{\log_2 7} + \left(1 + \frac{7a}{4} \right) n^2 < \textcolor{red}{cn^{\log_2 7}} \quad (1) \end{aligned}$$

3. Choose $a = -4$ then (1) holds.

Exercises

Estimate $T(n)$ in terms of a polynomial of n .

1. $T(n) = 2T(\sqrt{n}) + \log_2 n.$

2. $T(n) = kT\left(\frac{n}{k}\right) + n$, where $k > 1$.

3. $T(n) = 2T\left(\frac{n}{2}\right) + n \log n.$

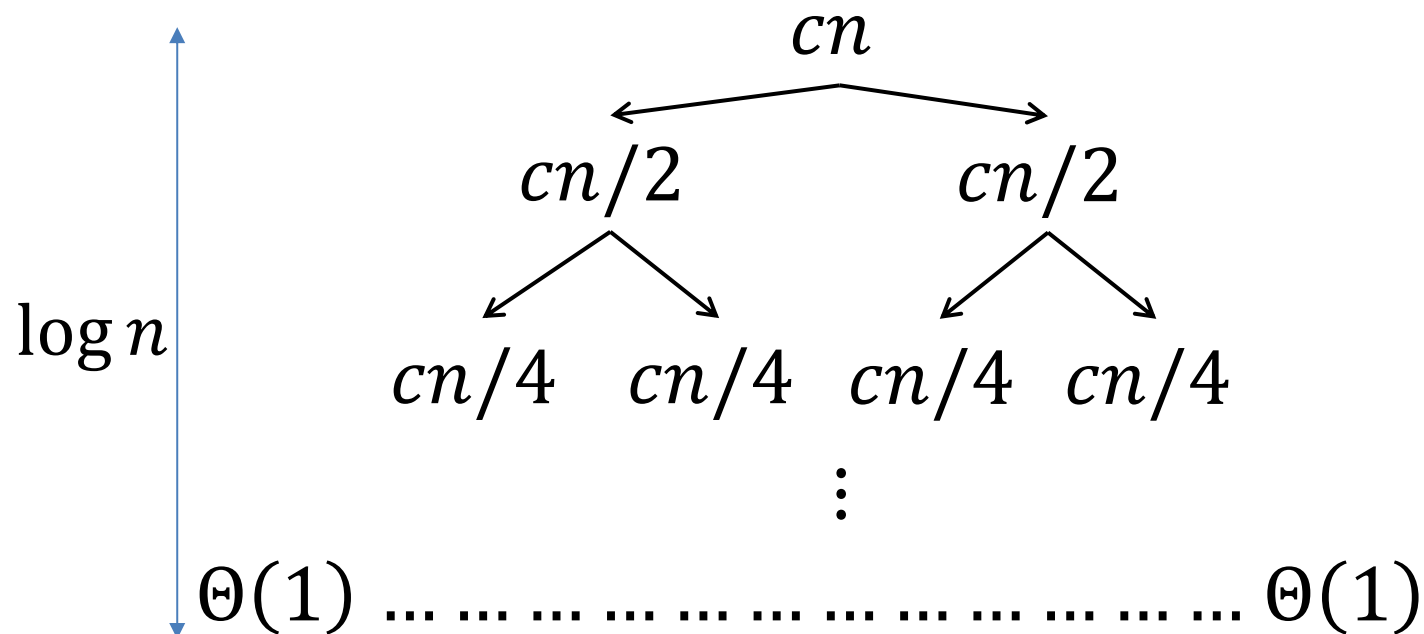
Outline

1. Substitution method
2. Recursion trees
3. Characteristic equation
4. The Master theorem

Example 1: Merge Sort

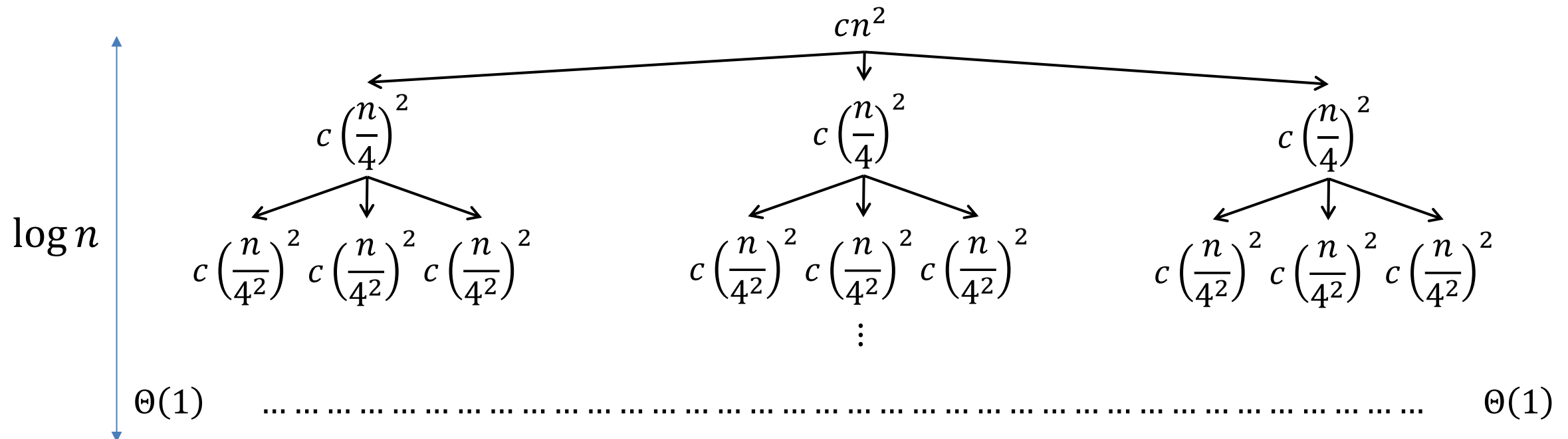
$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 2T\left(\frac{n}{2}\right) + \Theta(n), & \text{otherwise} \end{cases}$$

1. Depth of the leaves (the tree height): $\log n$.
2. Sum for each level: cn .
3. Sum across all levels: $cn \log n = O(n \log n)$.



Example 2: Advanced Merge Sort (1/2)

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 3T\left(\frac{n}{4}\right) + \Theta(n^2), & \text{otherwise} \end{cases}$$



Example 2: Advanced Merge Sort (2/2)

1. Depth of the leaves (the tree height): $\log n$.
2. Sum for each level: $3^i c (n/4^i)^2 = (3/16)^i c n^2$.
3. Sum across all levels:

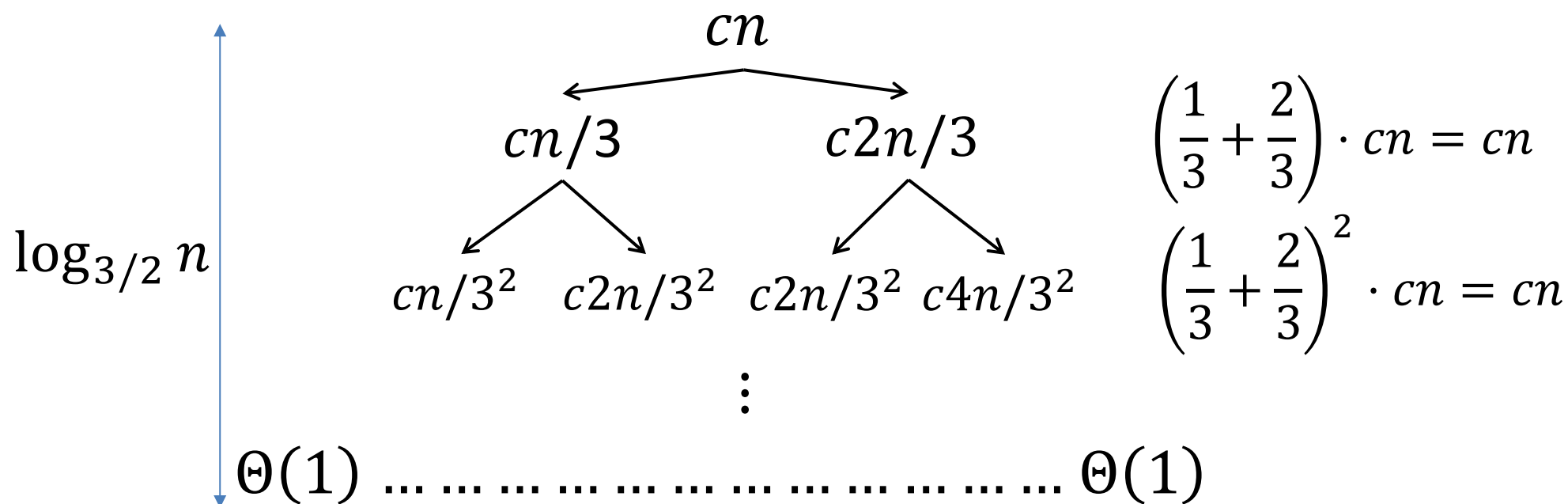
$$T(n) = \sum_{i=0}^{\log_4 n} \left(\frac{3}{16}\right)^i c n^2 \leq c n^2 \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i = \frac{1}{1 - 3/16} c n^2 = O(n^2)$$

Moreover, $T(n) = 3T\left(\frac{n}{4}\right) + \Theta(n^2) = \Omega(n^2)$ then $T(n) = \Theta(n^2)$.

Example 3

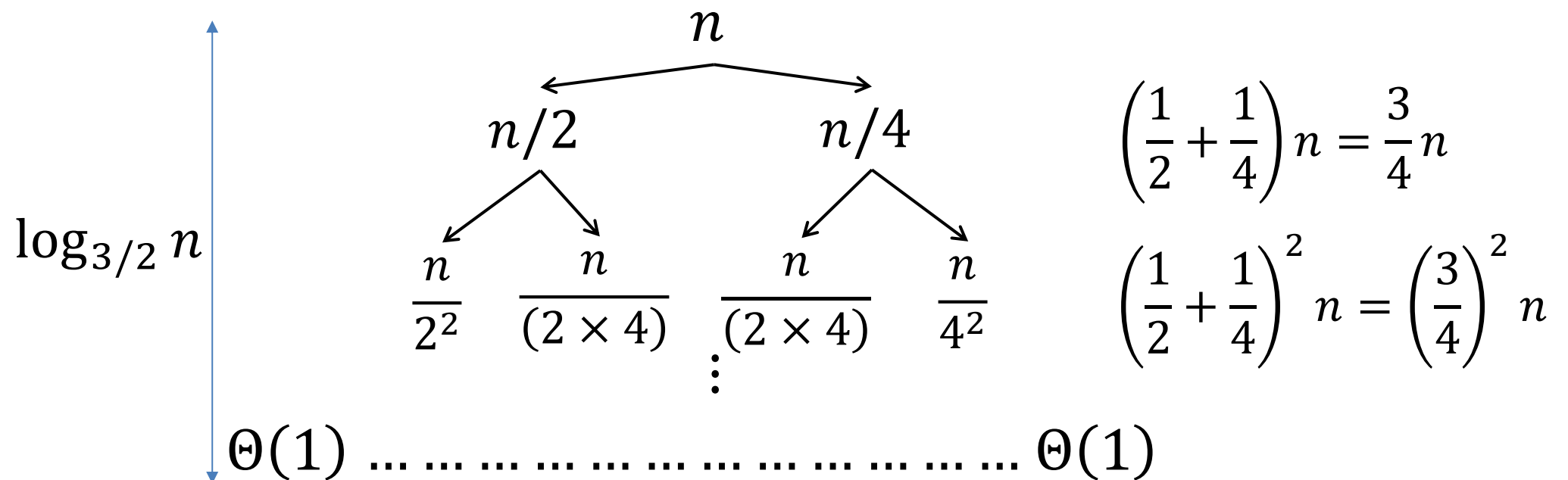
$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + O(n), & \text{otherwise} \end{cases}$$

1. Depth of the leaves (the tree height): $\log_{3/2} n$.
2. Sum for each level: $\leq cn$.
3. Sum across all levels: $cn \log_{3/2} n = O(n \log n)$.



Example 4 (1/2)

$$T(n) = \begin{cases} 1, & \text{if } n = 1 \\ T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + n, & \text{otherwise} \end{cases}$$



Example 4 (2/2)

1. Depth of the leaves (the tree height): $\log_4 n$.
2. Sum at level i : $(3/4)^i n$.
3. Sum across all levels:

$$T(n) = \sum_{i=0}^{\log_4 n} \left(\frac{3}{4}\right)^i n \leq n \sum_{i=0}^{\infty} \left(\frac{3}{4}\right)^i = \frac{1}{1 - 3/4} n = O(n)$$

Moreover, $T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{4}\right) + n = \Omega(n)$ then $T(n) = \Theta(n)$.

Outline

1. Substitution method
2. Recursion trees
3. Characteristic equation
4. The Master theorem

Recurrent relation: definition

1. A first- or second-order linear homogeneous recurrence relations

$$T(n) = \begin{cases} a \cdot T(n-1) + f(n) \\ a \cdot T(n-1) + b \cdot T(n-2) + f(n) \end{cases}$$

2. Their corresponding characteristic functions.

$$\begin{cases} x = a \\ x^2 = ax + b \end{cases}$$

Recurrent relation: definition

1. A first- or second-order linear homogeneous recurrence relations

$$T(n) = \begin{cases} a \cdot T(n-1) + f(n) \\ a \cdot T(n-1) + b \cdot T(n-2) + f(n) \end{cases}$$

2. Their corresponding characteristic functions.

$$\begin{cases} x = a \\ x^2 = ax + b \end{cases}$$

Solving the first-order characteristic equation

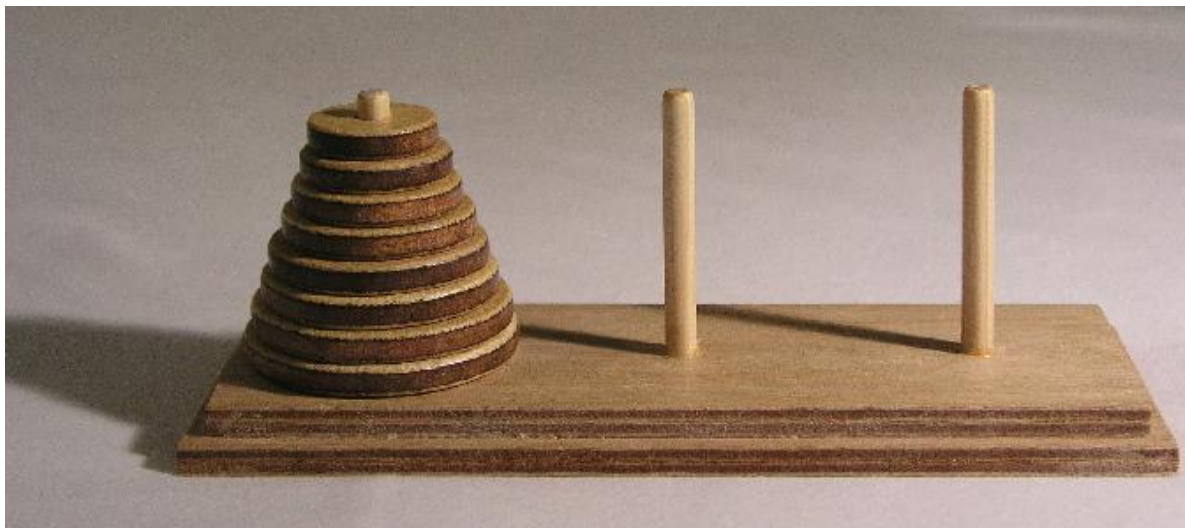
$$\begin{aligned}T(n) &= a \cdot T(n-1) + f(n) \\&= a(a \cdot T(n-2) + f(n-1)) + f(n) \\&= a^2 T(n-2) + af(n-1) + f(n) \\&= a^{n-1} T(1) + \sum_{i=0}^{n-2} a^i f(n-i)\end{aligned}$$

Example: Hanoi's Tower (1/6)

A mathematical puzzle consisting of three rods and a number of disks of various diameters, which can slide onto any rod. The puzzle begins with the disks stacked on one rod in order of decreasing size, the smallest at the top, thus approximating a conical shape. The objective of the puzzle is to move the entire stack to one of the other rods, obeying the following rules:

1. Only one disk may be moved at a time.
2. Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or an empty rod.
3. No disk may be placed on top of a disk that is smaller than it.

Example: Hanoi's Tower (2/6)



A model set of the Tower of Hanoi with 8 disks

Example: Hanoi's Tower (3/6)

According to an old Indian legend, the Brahmins have been following each other for a very long time on the steps of the alter in the Temple of Bernares, carrying out the moving of the Sacred Tower of Brahma with sixty-four levels in fine gold, trimmed with diamonds from Golconda. When all is finished, the Tower and the Brahmins will fall, and that will be the end of the world!

Example: Hanoi's Tower (4/6)

1. The minimal number of moves required to solve a Tower of Hanoi puzzle with n disks is $2^n - 1$. [1]
2. If the old Indian legend were true, and if the priests were able to move disks at a rate of one per second, using the smallest number of moves, it would take them $2^{64} - 1$ seconds or roughly 585 billion years to finish, which is about 42 times the estimated current age of the universe (14 billions years old).

Example: Hanoi's Tower (5/6): source code

Analyze the number of printings

```
void TowerOfHanoi(int n, char source, char target, char auxiliary) {
    if (n > 0) {
        // Move n-1 disks from source to auxiliary using target as the temporary peg
        TowerOfHanoi(n - 1, source, auxiliary, target);

        // Move the nth disk from source to target
        cout << "Move disk " << n << " from " << source << " to " << target << endl;

        // Move n-1 disks from auxiliary to target using source as the temporary peg
        TowerOfHanoi(n - 1, auxiliary, target, source);
    }
}
```

$$\begin{aligned}
 &T(n) \\
 &= \\
 &T(n - 1) \\
 &+ \\
 &1 \\
 &+ \\
 &T(n - 1)
 \end{aligned}$$

Example: Hanoi's Tower (6/6)

1. We have

$$T(n) = 2T(n - 1) + 1$$

$$\Rightarrow T(1) = 1, a = 2, \text{ and } f(n) = 1.$$

$$\Rightarrow T(n)$$

$$= a^{n-1}T(1) + \sum_{i=0}^{n-2} a^i f(n - i) = 2^{n-1} \cdot 1 + \sum_{i=0}^{n-2} 2^i = 2^{n-1} + \frac{(2^{n-1} - 1)}{2 - 1} = 2^n - 1$$

Recurrent relation: definition

1. A first- or second-order linear homogeneous recurrence relations

$$T(n) = \begin{cases} a \cdot T(n-1) + f(n) \\ \textcolor{red}{a \cdot T(n-1) + b \cdot T(n-2) + f(n)} \end{cases}$$

2. Their corresponding characteristic functions.

$$\begin{cases} x = a \\ \textcolor{red}{x^2 = ax + b} \end{cases}$$

Recurrent relation: solving the second-order characteristic equation

Let x_1 and x_2 be the roots of $x^2 = ax + b$. Then $\alpha + \beta = a$ and $\alpha\beta = b$. Then we have:

$$T(n) = a \cdot T(n-1) + b \cdot T(n-2) + f(n) = (\alpha + \beta)T(n-1) + \alpha\beta T(n-2) + f(n)$$

$$\Leftrightarrow T(n) - \alpha T(n-1) = \beta(T(n-1) - \alpha T(n-2)) + f(n)$$

$$= \beta(\beta(T(n-2) - \alpha T(n-3)) + f(n-1)) + f(n)$$

$$= \beta^2(T(n-2) - \alpha T(n-3)) + \sum_{i=0}^1 \beta^i f(n-i)$$

$$= \beta^{n-1}(T(1) - \alpha T(0)) + \sum_{i=0}^{n-2} \beta^i f(n-i) = \beta^{n-1}C + \sum_{i=0}^{n-2} \beta^i f(n-i)$$

Recurrent relation: solving the second-order characteristic equation

$$T(n) - \alpha T(n-1) = \beta^{n-1} C + \sum_{i=0}^{n-2} \beta^i f(n-i)$$

$$\Rightarrow T(n) = \alpha^{n-1} T(1) + C \sum_{j=0}^{n-2} \alpha^j \beta^{n-1-j} + \sum_{j=0}^{n-2} \left(\alpha^j \sum_{i=0}^{n-2-j} \beta^i f(n-j-i) \right)$$

Example: Fibonacci

1. The Fibonacci sequence is a sequence in which each number is the sum of the two preceding ones.

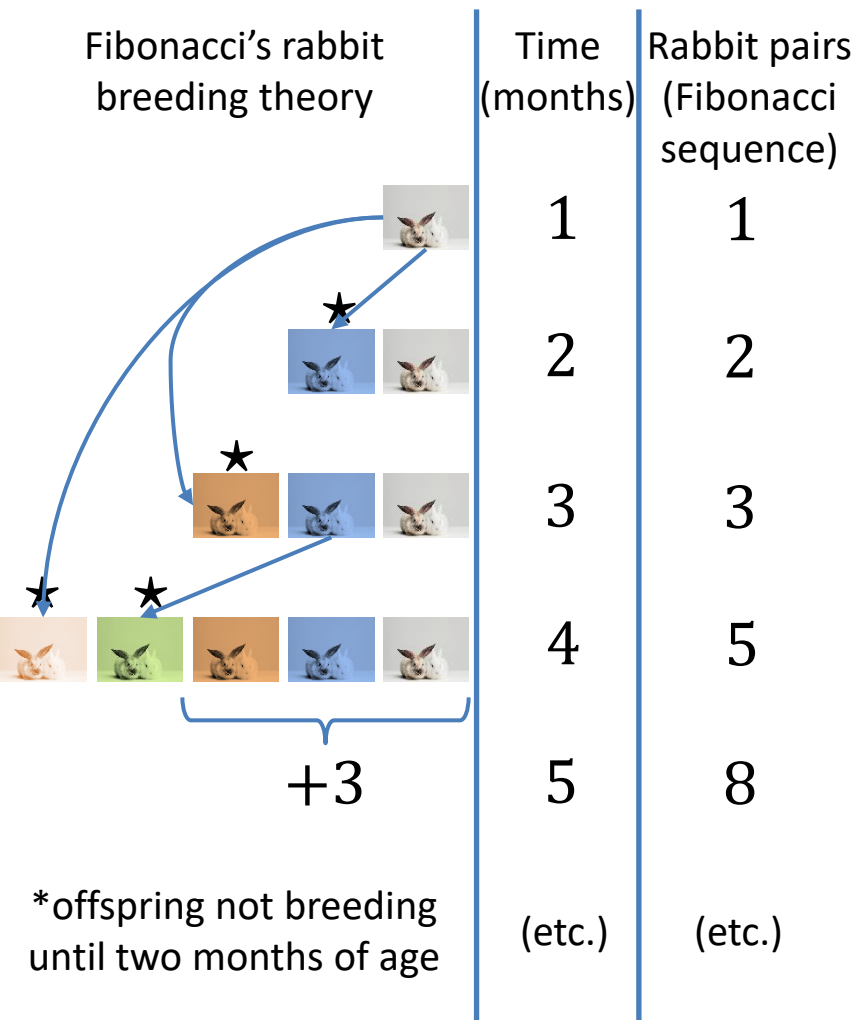
$$F(n) = F(n - 1) + F(n - 2) \text{ and } F(0) = 0 \text{ and } F(1) = 1$$

2. The Fibonacci numbers were first described in **Indian mathematics** as early as **200 BC** in a work by Pingala on enumerating possible patterns of Sanskrit poetry formed from syllables of two lengths.
3. Introduced to Western European mathematics in Leonardo of Pisa's book titled *Liber Abaci* in 1202 [2].

[1] Singh, Parmanand. "The so-called Fibonacci numbers in ancient and medieval India." *Historia Mathematica* 12.3 (1985): 229-244.

[2] Sigler, Laurence. *Fibonacci's Liber Abaci: a translation into modern English of Leonardo Pisano's book of calculation*. Springer Science & Business Media, 2003.

Example: Fibonacci in Animal Breeding Patterns



1. Rabbits follow the Fibonacci sequence in their reproductive pattern.
2. A single pair of rabbits (one male and one female) produces another pair in the second month.
3. From the third month onwards, each pair produces a new pair every month.
4. The sequence of pairs of rabbits is 1, 1, 2, 3, 5, 8, 13, and so on.
5. The breeding pattern reflects the natural balance of reproduction and population growth.
6. The population grows slowly initially as the breeding pairs are young and need time to mature.
7. As the population increases, more pairs are available to reproduce, leading to an exponential growth rate that follows the Fibonacci sequence.

Example: Fibonacci numbers: source code

Analyze the number of returns

```
int Fibonacci(int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
  
    return Fibonacci(n-1) + Fibonacci(n-2);  
}
```

 $T(n)$ $=$ $T(n-1) + T(n-2)$

Example: Fibonacci numbers (1/2)

1. We have

$$T(n) = T(n - 1) + T(n - 2)$$

$\Rightarrow \alpha = \frac{1+\sqrt{5}}{2}, \beta = \frac{1-\sqrt{5}}{2}$ which are roots of $x^2 = x + 1$ and

$$f(n) = 0$$

Example: Fibonacci numbers (2/2)

$$\begin{aligned}
 T(n) &= \alpha^{n-1}T(1) + (T(1) - \alpha T(0)) \sum_{j=0}^{n-2} \alpha^j \beta^{n-1-j} = \alpha^{n-1} + \beta^{n-1} \sum_{j=0}^{n-2} \left(\frac{\alpha}{\beta}\right)^j \\
 &= \alpha^{n-1} + \beta^{n-1} \cdot \frac{1 - \left(\frac{\alpha}{\beta}\right)^{n-1}}{1 - \frac{\alpha}{\beta}} = \alpha^{n-1} + \frac{\beta^n}{\beta - \alpha} - \frac{\beta}{\beta - \alpha} \alpha^{n-1} \\
 &= \frac{\alpha^n - \beta^n}{\alpha - \beta} \\
 &= \frac{1}{\sqrt{5}} \left(\left(\frac{1 + \sqrt{5}}{2} \right)^n - \left(\frac{1 - \sqrt{5}}{2} \right)^n \right)
 \end{aligned}$$

Exercise

Estimate $T(n) = 2T(n-1) - 4T(n-2)$.

- Let $\alpha = 1 + i\sqrt{3} = 2\left(\cos\frac{\pi}{3} + i\sin\frac{\pi}{3}\right)$. Then, we have:
- $$\begin{aligned}
 T(n) - \alpha T(n-1) &= (2 - \alpha)(T(n-1) - \alpha T(n-2)) \\
 &= (2 - \alpha)^2(T(n-2) - \alpha T(n-3)) \\
 &= \dots \\
 &= (2 - \alpha)^{n-2}(T(2) - \alpha T(1)) \\
 &= (2 - \alpha)^{n-2}T(2) - \alpha(2 - \alpha)^{n-2}T(1)
 \end{aligned}$$
- $$\begin{aligned}
 \Rightarrow T(n) - \alpha^{n-1}T(1) &= \sum_{k=0}^{n-2} \alpha^k \left((2 - \alpha)^{n-2-k}T(2) - \alpha(2 - \alpha)^{n-2-k}T(1) \right) \\
 \Rightarrow T(n) &= \frac{\alpha^{n-1} - (2 - \alpha)^{n-1}}{\alpha - (2 - \alpha)} T(2) - \alpha(2 - \alpha) \frac{\alpha^{n-2} - (2 - \alpha)^{n-2}}{\alpha - (2 - \alpha)} T(1) \\
 &= \frac{2^{n-1}}{\sqrt{3}} \sin \frac{(n-1)\pi}{3} T(2) - \frac{2^n}{\sqrt{3}} \sin \frac{(n-2)\pi}{3} T(1) = \Theta(2^n)
 \end{aligned}$$

Outline

1. Substitution method
2. Recursion trees
3. Characteristic equation
4. The Master theorem

Theorem

Consider the recurrence

$$T(n) = aT(n/b) + f(n)$$

1. If there exists $\epsilon > 0$ for which $f(n) = O(n^{\log_b a - \epsilon})$ then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$ then $T(n) = \Theta(n^{\log_b a} \log n)$.
3. If there exists $\epsilon > 0$ for which $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af(n/b) \leq cf(n)$ for some constant c and sufficient large n then $T(n) = \Theta(f(n))$.

Example 1

$$T(n) = 9T\left(\frac{n}{3}\right) + n$$

Example 2

$$T(n) = T\left(\frac{2n}{3}\right) + 1$$

Example 3

$$T(n) = 3T\left(\frac{n}{4}\right) + n \log n$$

Example 4 (1/2)

$$T(n) = 2T\left(\frac{n}{2}\right) + n \log n$$

Exercise 1: Karatsuba's algorithm

1. Input: Two positive number a, b .
2. Output: The product $f(a, b) = a \times b$

Compute the complexity of the following pseudocode:

- Step 1. If both a and b have only one binary digit, then return $a \times b$.
- Step 2. $n \leftarrow \max\{\lceil \log_2 a \rceil, \lceil \log_2 b \rceil\}$.
- Step 3. Split a into 2 parts (a_l, a_h) where a_l contains $\lfloor \frac{n}{2} \rfloor$ lower binary digits, and a_h contains $\lceil \log_2 a \rceil - \lfloor \frac{n}{2} \rfloor$ higher binary digits. Split b into 2 parts (b_l, b_h) where b_l contains $\lfloor \frac{n}{2} \rfloor$ lower binary digits, and b_h contains $\lceil \log_2 b \rceil - \lfloor \frac{n}{2} \rfloor$ higher binary digits.
- Step 4. $z_0 \leftarrow f(a_l, b_l)$; $z_1 \leftarrow f(a_h + a_l, b_h + b_l)$; $z_2 \leftarrow f(a_h, b_h)$.
- Step 5. Return $z_2 \times 2^{2\lfloor \frac{n}{2} \rfloor} + (z_1 - z_2 - z_0) \times 2^{\lfloor \frac{n}{2} \rfloor} + z_0$

Exercise 2: Strassen's algorithm

Input: Two matrices $A, B \in \mathbb{R}^{n \times n}$

Output: The product $A \times B$.

The idea:

- $$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix}, C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix} = A \times B$$

$$M_1 = (A_{11} + A_{22}) \times (B_{11} + B_{22})$$

$$M_2 = (A_{21} + A_{22}) \times B_{11}$$

$$M_3 = A_{11} \times (B_{12} - B_{22})$$

$$M_4 = A_{22} \times (B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{12}) \times B_{22}$$

$$M_6 = (A_{21} - A_{11}) \times (B_{11} + B_{12})$$

$$M_7 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$$

- Then, we have:

$$C_{11} = M_1 + M_4 - M_5 + M_7$$

$$C_{12} = M_3 + M_5$$

$$C_{21} = M_2 + M_4$$

$$C_{22} = M_1 - M_2 + M_3 + M_6$$

- Let $T(n)$ be the number of multiplications and additions two elements of matrices. Estimate $T(n)$.

Q & A